University of North Florida

School of Computing

COP3503

Project 2 – Road Section Data

Objective:

- Show an understanding of how to
  - Use an input stream to read in data from a delimited text file
  - Create object class files based on input files and output requirements
  - Use existing methods to convert dates into different formats
  - Use an output stream to create a delimited text file
  - Catch and recover from exceptions that can occur

Submission:

Project 2 requires that you submit 1 zip file to Canvas as specified below.

- Submit your project via the Canvas course
  - Upload a zip file containing the 6 files required for the project. Follow the naming convention below.
    - Flowchart.jpg/png
    - Project2.java
    - FileHandler.java
    - RoadVolume.java
    - RoadSpeed.java
    - RoadSection.java
  - The zip file should be named Project2_N#.zip
    - For example: Project2_n00123456.zip
  - The zip file should not contain any additional files
    - <u>This includes input and output files</u>
  - The files should be named as shown above
    - Only the zip file name should contain your N#
  - After submitting to Canvas download and double check submission
    - No late files will be taken after the submission date

Description:

The Road Section Data program combines volume and speed data for a given section of roadway. The program will read in data from two files one containing speed data and the other volume data. The data sets must be linked together based on their date and time attributes. This will allow for a new data set to be created that stores the related volume and speed data along with the total volume and average volume/speed for a given date and time.

Requirements:

Documentation Requirements: 10 points

- 10 points: Create a flowchart that outlines the logic implemented in the createRoadSections method
    - You must use a flowcharting program to create this diagram – NO Hand drawn diagrams
        - Free Recommended option draw.io
    - Make sure to export the flowchart as an image from the flowcharting program.
        - DO NOT take a screenshot or submit the draw.io file
    - Create the flowchart after completing the code for Project 2. The diagram must accurately reflect your implementation of project 2

Minimum Implementation Requirements:-100 points

The requirements listed below must be implemented correctly or no points will be given for the project.

- Must use at least 1 try-catch in your project
    - Your try-catch/s should catch, handle, and recover from FileNotFoundExceptions, and ParseExceptions
- All 5 class files must be present in the submission
    - Project2.java, FileHandler.java, RoadSpeed.java, RoadVolume.java, & RoadSection.java
- All methods and variables shown in the UML class diagrams must be present in each class file
    - See below for UML class diagram
- Program must read in the path/filename to the Volume Date file then the Speed Data file

Failure to meet the requirements listed above will result in a zero for the project.

Style Requirements: Required

Javadoc's: -10 points

- Each class and method declaration must have a correct Javadoc comment placed above it
    - Add your Javadoc comments last
    - Generate the Javadoc's using an IDE and verify no errors or warning are produced

Naming Conventions: -10 points

- Each class, variable and method must be named appropriately
    - Variables and methods always start with a lowercase letter
    - Classes always start with an uppercase letter

Implementation Requirements: 90 points

Prompt User for Input Files:

- Prompt the user to "Enter Path and Name of Volume and Speed Data File" on a single line
    - The paths/filenames will be space delimited
- The first path given will always be the path/filename Volume Data file
- The second path given will always be the path/filename Speed Data file
- THE PROGRAM MUST READ THE FILES IN THIS ORDER – Volume then Speed

Load Volume Data:

Use the method in FileHandler called loadVolumeData to handle this task.

- Before calling the loadVolumeData method print to the console "Loading Volume Data"
- If the input given by the user results in a FileNotFoundException alert the user to the specific error and re-prompt them for good input
    - The user providing an invalid path or filename should not cause the program to crash or stop running
    - Recommend setting the loadVolumeData method to throw the FileNotFoundException
        - Catch and recover from the exception in the calling method that takes in the filename
- Read in the data from the provided volume data file and store the data for each row in a RoadVolume object. Store each object created in an ArrayList.
    - If the input file given by the user contains bad date data and results in a ParseException alert the user to the specific error and re-prompt them for good input
    - The user providing a file with bad date data should not cause the program to crash or stop running
    - Recommend setting the loadVolumeData method to throw the ParseException
        - Catch and recover from the exception in the calling method that takes in the filename
    - If the input file given by the user contains bad number data and results in a NumberFormatException alert the user to the specific error and re-prompt them for good input
    - The user providing a file with bad number data should not cause the program to crash or stop running
    - Recommend setting the loadVolumeData method to throw the NumberFormatException
        - Catch and recover from the exception in the calling method that takes in the filename
- After the loadVolumeData method is called print to the console "Volume Data Loaded"

Load Speed Data:

Use the method in FileHandler called loadSpeedData to handle this task.

- Before calling the loadSpeedData method print to the console "Loading Speed Data"
- If the input given by the user results in a FileNotFoundException alert the user to the specific error and re-prompt them for good input
    - The user providing an invalid path or filename should not cause the program to crash or stop running
    - Recommend setting the loadSpeedData method to throw the FileNotFoundException
        - Catch and recover from the exception in the calling method that takes in the filename
- Read in the data from the provided volume data file and store the data for each row in a RoadSpeed object. Store each object created in an ArrayList.

    - If the input file given by the user contains bad date data and results in a ParseException alert the user to the specific error and re-prompt them for good input
    - The user providing a file with bad date data should not cause the program to crash or stop running
    - Recommend setting the loadSpeedData method to throw the ParseException
        - Catch and recover from the exception in the calling method that takes in the filename
    - If the input file given by the user contains bad number data and results in a NumberFormatException alert the user to the specific error and re-prompt them for good input
    - The user providing a file with bad number data should not cause the program to crash or stop running
    - Recommend setting the loadSpeedData method to throw the NumberFormatException
        - Catch and recover from the exception in the calling method that takes in the filename
- After the loadSpeedData method is called print to the console "Speed Data Loaded"

Create Road Section Objects:

Use the method in Project2 called createRoadSections to handle this task.

- For each volume object find the matching speed object based on the date and time attributes
    - This can be done in reverse as well
        - Each speed object has a single volume object that contains a matching date/time
- Once a matching volume and speed object has been found create a RoadSection object that stores references to the matching volume and speed objects
    - Each RoadSection object created needs to be stored in an ArrayList
- When the RoadSection object is created calculate the volume total, volume average, and speed average based on the data stored in the matching objects
    - Use the private methods in the RoadSection to calculate the values mentioned above

Write Road Section Data:

Use the method in FileHandler called writeRoadSectionData to handle this task. Along with the getFileData methods of each object.

getFileData Method:

- The getFileData for the volume and speed data objects need to return their respective sensor data comma separated
- The RoadSection getFileData method needs to leverage the getFileData methods of the volume and speed objects while appending the additional data needed
- The string returned by RoadSection getFileData must be comma separated and contain the values for the attributes listed below

- Date,Time,Volume_Sensor_2003,Volume_Sensor_2004,Volume_Sensor_2005,Volume_Sensor_2006,Speed_Sensor_2282,Speed_Sensor_2293,Volume_Total,Volume_Avg,Speed_Avg
- The output of the Date string must match the format MM/dd/yyyy

writeRoadSectionData Method:

- Create a new csv file called "Road_Section_Data.csv"
- Write out the following header to the csv file
  - Header:
    - Date,Time,Volume_Sensor_2003,Volume_Sensor_2004,Volume_Sensor_2005,Volume_Sensor_2006,Speed_Sensor_2282,Speed_Sensor_2293,Volume_Total,Volume_Avg,Speed_Avg
- Then for each RoadSection object created call getFileData and write the returned string to the new csv file

Class Diagrams:

Project2:

Project2
+ main(args : String[]) : void + createRoadSections(volumeList : ArrayList<RoadVolume> ,speedList : ArrayList<RoadSpeed>) : ArrayList<RoadSection>

FileHandler:

FileHandler
+ loadVolumeData(filename:String) : ArrayList<RoadVolume> + loadSpeedData(filename:String) : ArrayList<RoadSpeed> + writeRoadSectionData(sectionList : ArrayList<RoadSection>) : void

RoadVolume:

RoadVolume
- date : Date - time : String - volumeSensor1 : int - volumeSensor2 : int - volumeSensor3 : int - volumeSensor4 : int
+ RoadVolume(date:Date, time:String, volumeSensor1:int, volumeSensor2:int,volumeSensor3:int, volumeSensor4:int) + getFileData() : String + getDate() : Date + setDate(date : Date) : void + getTime() : String + setTIme(time : String) : void + getVolumeSensor1() : int + setVolumeSensor1(volumeSensor1: int) : void + getVolumeSensor2() : int + setVolumeSensor2(volumeSensor2: int) : void + getVolumeSensor3() : int + setVolumeSensor3(volumeSensor3: int) : void + getVolumeSensor4() : int + setVolumeSensor4(volumeSensor4: int) : void

RoadSpeed:

RoadSpeed
- date : Date - time : String - speedSensor1 : double - speedSensor2 : double
+ RoadSpeed(date:Date, time:String, speedSensor1:double, speedSensor2:double) + getFileData() : String + getDate() : Date + setDate(date : Date) : void + getTime() : String + setTIme(time : String) : void + getSpeedSensor1() : double + setSpeedSensor1(speedSensor1 : double) : void + getSpeedSensor2() : double + setSpeedSensor2(speedSensor2 : double) : void

RoadSection:

RoadSection
- roadVolume : RoadVolume - roadSpeed : RoadSpeed - volumeTotal : int - volumeAvg : double - speedAvg : double
+ RoadSection(roadVolume : RoadVolume, roadSpeed : RoadSpeed) + getFileData() : String + getRoadVolume() : RoadVolume + setRoadVolume(roadVolume: RoadVolume) : void + getRoadSpeed() : RoadSpeed + setRoadSpeed(roadSpeed: RoadSpeed) : void + getVolumeTotal() : int + setVolumeTotal(total : int) : void + getVolumeAvg() : double + setVolumeAvg(avg : double) : void + getSpeedAvg() : double + setSpeedAvg(avg : double) : void - calcVolumeTotal() : int - calcVolumeAvg() : double - calcSpeedAvg() : double

Test Cases:

The submitted program must pass all 3 test cases outlined below. When developing and testing your program use the input files given.

Test Case 1: FileNotFound

Overview: Test if your program successfully catches FileNotFound exceptions for Volume Data and Speed Data. Then verify that once given good input the program runs successfully and creates the correct output file.

```
Enter Path and Name of Volume and Speed Data File
NotAValidFileOrPath.csv Speed_Data.csv
Loading Volume Data

File Not Found Error: NotAValidFileOrPath.csv (The system cannot find the file
specified)

Enter Path and Name of Volume and Speed Data File
```

```
Volume_Data.csv NotAValidFileOrPath.csv
Loading Volume Data
Volume Data Loaded
Loading Speed Data

File Not Found Error: NotAValidFileOrPath.csv (The system cannot find the file
specified)

Enter Path and Name of Volume and Speed Data File
Volume_Data.csv Speed_Data.csv
Loading Volume Data
Volume Data Loaded
Loading Speed Data
Speed Data Loaded
Road Section Data Created
```

Test Case 2: ParseException

Overview: Test if your program successfully catches Parse exceptions for Volume Data and Speed Data. Then verify that once given good input the program runs successfully and creates the correct output file.

```
Enter Path and Name of Volume and Speed Data File
Volume_Data_Bad_Date.csv Speed_Data.csv
Loading Volume Data
Date Parse Error: Unparseable date: "020715"

Enter Path and Name of Volume and Speed Data File
Volume_Data.csv Speed_Data_Bad_Date.csv
Loading Volume Data
Volume Data Loaded
Loading Speed Data
Date Parse Error: Unparseable date: "2015099"

Enter Path and Name of Volume and Speed Data File
Volume_Data.csv Speed_Data.csv
Loading Volume Data
Volume Data Loaded
Loading Speed Data
Speed Data Loaded
Road Section Data Created
```

Test Case 3: NumberFormatException

Overview: Test if your program successfully catches Number Format exceptions for Volume Data and Speed Data. Then verify that once given good input the program runs successfully and creates the correct output file.

```
Enter Path and Name of Volume and Speed Data File
Volume_Data_Bad_Num.csv Speed_Data.csv
Loading Volume Data
Number Parse Error: For input string: "12g5"

Enter Path and Name of Volume and Speed Data File
Volume_Data.csv Speed_Data_Bad_Num.csv
```

```
Loading Volume Data
Volume Data Loaded
Loading Speed Data
Number Parse Error: For input string: "5B.5"

Enter Path and Name of Volume and Speed Data File
Volume_Data.csv Speed_Data.csv
Loading Volume Data
Volume Data Loaded
Loading Speed Data
Speed Data Loaded
Road Section Data Created
```

Test Case 4: Road Section Output

Overview: Test if your program successfully executes when given good input and that it creates the correct output file.

```
Enter Path and Name of Volume and Speed Data File
Volume_Data.csv Speed_Data.csv
Loading Volume Data
Volume Data Loaded
Loading Speed Data
Speed Data Loaded
Road Section Data Created
```